

RIXI LAB TECHNOLOGY
FULL STACK DEVELOPMENT INTERNSHIP
PROJECT REPORT

Week 2 — Project 2

A Full-Stack Personal Portfolio Website with Admin Content Management

Project Code: FS07P2B

By

Name: Bipin Kumar Pandey

Role: Intern

Course: B. Tech CSE

DECLARATION

I hereby declare that the project report entitled "Portfolio: A Full-Stack Personal Portfolio Website with Admin Content Management" submitted to Rixi Lab Technologies in partial fulfilment of the requirements for the award of the Summer Internship is a record of original work done by me, under the guidance and supervision of Mentor Sanskriti Singh.

Dated: 18/07/2026

CERTIFICATE

It is certified that the work contained in the project report titled "Portfolio: A Full-Stack Personal Portfolio Website with Admin Content Management," by "Bipin Kumar Pandey," has been carried out under my/our supervision and that this work has not been submitted elsewhere for a degree.

Mentor: Sanskriti Singh

Rixi Lab Technologies

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Sanskriti Singh, my project mentor, for her invaluable guidance, continuous support, and encouragement throughout the duration of this project. Her insights and suggestions were instrumental in shaping the direction and quality of this work.

I am deeply thankful to Rixi Lab Technology for providing the internship along with the necessary facilities, infrastructure, and a conducive learning environment that made this project possible.

I extend my heartfelt thanks to my family and friends for their constant motivation and moral support during the course of this project.

Finally, I would like to acknowledge the open-source community and the documentation of React, Vite, Tailwind CSS, Node.js, Express, and MongoDB (Mongoose), which served as valuable references during development.

Bipin Kumar Pandey

ABSTRACT

A personal portfolio website is one of the most effective ways for a developer to present their skills, projects, and contact information to the world, and building one independently is a widely used exercise for demonstrating full-stack web development competency. This project presents Portfolio, a full-stack personal portfolio website consisting of a public-facing site and a secure admin panel, built using React (with Vite) and Tailwind CSS on the frontend and Express.js with MongoDB (via Mongoose) on the backend, with JSON Web Tokens (JWT) used to protect all administrative operations.

The application is architected as a decoupled client-server system: the React single-page application communicates with the Express backend exclusively through a REST/JSON API. Content is organised into three independent MongoDB collections — Project, Skill, and Message — each with its own Mongoose schema. The public site allows visitors to browse featured projects, view categorised skills, and submit a contact message, while the admin panel allows the site owner to log in and perform full CRUD (Create, Read, Update, Delete) operations on projects and skills, as well as review incoming contact messages.

The project demonstrates core full-stack concepts including RESTful API design, stateless JWT-based authentication, MongoDB schema modelling with Mongoose, component-driven frontend development with React, and utility-first styling with Tailwind CSS.

TABLE OF CONTENTS

Declaration.....	i
Certificate.....	ii
Acknowledgement	iii
Abstract.....	iv
Chapter	
1. Introduction	1
2. System Requirements	2
3. System Design	4
4. Implementation	7
5. Testing.....	9
6. Results and Discussion	10
7. Advantages and Limitations.....	11
8. Future Enhancements.....	12
9. Conclusion	13
10. References	14
11. Source Code	15

LIST OF ABBREVIATIONS

Abbreviation	Full Form
UI	User Interface
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
JWT	JSON Web Token
DB	Database
CRUD	Create, Read, Update, Delete
SPA	Single-Page Application

1. INTRODUCTION

1.1 Background

A strong online presence is essential for developers seeking internships, freelance work, or full-time roles, and a personal portfolio website is the most direct way to showcase projects, skills, and contact details to prospective employers or clients. Static portfolio websites, hand-coded in plain HTML, are simple to build but become tedious to maintain, since every new project or skill added requires editing source code and redeploying the site.

This project was undertaken to design and implement Portfolio, a full-stack personal portfolio website where projects and skills can be added, edited, or removed through a dedicated admin panel, without ever touching the underlying code. The project was built as a decoupled client-server application, with a React single-page application on the frontend communicating with an Express REST API on the backend.

1.2 Problem Statement

Developers frequently need to keep their portfolio content current, but rebuilding and redeploying a static site for every content change is inefficient and error-prone. This project addresses that by implementing a full-stack web application consisting of a public-facing portfolio site and a secure, authenticated admin panel, where the site owner can manage projects, skills, and incoming contact messages entirely through a browser-based interface, backed by a REST API and a persistent database.

1.3 Objectives

1. To design MongoDB schemas (via Mongoose) for a project, a skill, and a contact message, each as an independent collection.
2. To implement a REST/JSON API using Express, exposing public read endpoints and JWT-protected write endpoints.
3. To build a responsive public-facing single-page application using React, Vite, and Tailwind CSS.
4. To implement a secure admin authentication flow using JWT, protecting all create, update, and delete operations.
5. To build an admin dashboard supporting full CRUD on projects and skills, and management of contact messages.

1.4 Scope of the Project

- Single-admin content management (no multi-user/team support).
- Full CRUD operations on projects and skills via a REST API, protected by JWT authentication.
- A public contact form that stores messages in the database for the admin to review.
- Categorisation of skills (Frontend, Backend, Mobile, Design, Tools) with a colour-coded display.

- A public portfolio site (home, projects, contact) and a separate authenticated admin panel.

2. SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Minimum Requirement
Processor	Dual-core 1.6 GHz or higher
RAM	4 GB (8 GB recommended)
Storage	500 MB free disk space
Internet Connection	Required (MongoDB Atlas, npm packages, CDN assets)

2.2 Software Requirements

Component	Requirement
Operating System	Windows / Linux / macOS
Runtime	Node.js v18 or later
Package Manager	npm
Database	MongoDB (Atlas cloud cluster or local instance)
Browser	Any modern browser (Chrome, Firefox, Edge)
Code Editor	Visual Studio Code (recommended)
Version Control	Git

2.3 Technology Stack

Layer	Technology	Purpose
Frontend Build Tool	Vite	Fast development server and bundler
Frontend Library	React	Component-based UI construction
Styling	Tailwind CSS	Utility-first responsive styling
Backend Runtime	Node.js	JavaScript server runtime
Web Framework	Express.js	REST API routing and middleware
Database	MongoDB	Document-oriented data persistence
ODM	Mongoose	Schema modelling and validation

Layer	Technology	Purpose
Authentication	JSON Web Tokens (JWT)	Stateless admin authentication

3. SYSTEM DESIGN

3.1 System Architecture

The client (React SPA) communicates with the Express REST API over HTTPS. Incoming requests pass through a router, an optional JWT authentication middleware for protected routes, and a controller, which reads or writes data through a Mongoose model to MongoDB. The response is returned as JSON and rendered by the React frontend.

Browser (React SPA) → Express Router → JWT Auth Middleware (protected routes only) → Controller → Mongoose Model → MongoDB (Atlas)

3.2 Database Design

The database consists of three independent MongoDB collections. There are no foreign keys or relations between them, since projects, skills, and messages are managed and queried entirely independently.

Project Schema (Collection: projects)

Field	Type	Notes
_id	ObjectId	Auto-generated primary key
title	String	Required
description	String	Required
imageUrl	String	Optional
githubLink	String	Optional
liveLink	String	Optional
techStack	[String]	Array of technology tags
createdAt	Date	Default: current time

Skill Schema (Collection: skills)

Field	Type	Notes
_id	ObjectId	Auto-generated primary key
name	String	Required
color	String	Enum: blue, green, amber, red, purple, teal
category	String	Enum: Frontend, Backend, Mobile, Design, Tools

Message Schema (Collection: messages)

Field	Type	Notes
_id	ObjectId	Auto-generated primary key
name	String	Required
email	String	Required, validated
subject	String	Required
message	String	Required
readStatus	Boolean	Default: false

3.3 API Design

Base URL (local): `http://localhost:5000/api`

Method	Endpoint	Auth	Purpose
GET	<code>/projects</code>	No	List all projects
POST	<code>/projects</code>	Yes (JWT)	Create project
PUT	<code>/projects/:id</code>	Yes (JWT)	Update project
DELETE	<code>/projects/:id</code>	Yes (JWT)	Delete project
GET	<code>/skills</code>	No	List all skills
POST	<code>/skills</code>	Yes (JWT)	Create skill
POST	<code>/messages</code>	No	Submit contact form
GET	<code>/messages</code>	Yes (JWT)	List messages
POST	<code>/auth/login</code>	No	Admin login, returns JWT

Protected routes require an `Authorization: Bearer <token>` header, validated by the authentication middleware before the request reaches the controller. Invalid or missing tokens receive a 401 response before any database access occurs.

3.4 Folder Structure

`client/src` — components, pages (Home, Projects, Contact, AdminLogin, AdminDashboard), context (JWT auth), and API client functions.

`server` — `config/db.js`, models (Project.js, Skill.js, Message.js), controllers, routes, `middleware/authMiddleware.js`, and `server.js` as the entry point.

4. IMPLEMENTATION

4.1 Backend Implementation

The entry point, `server.js`, configures Express with JSON body parsing, CORS, and mounts all route modules under `/api`. It defines a 404 handler for unmatched routes and centralised error-handling middleware that returns a structured JSON error response instead of crashing. `app.listen()` is only called after the database connection succeeds.

4.2 Authentication (JWT)

The auth controller exposes a login endpoint that validates admin credentials and, on success, signs a JWT using a secret stored in an environment variable. The authentication middleware reads the Authorization header on protected routes, verifies the token, and attaches the decoded admin identity to the request; invalid or missing tokens are rejected with a 401 response.

4.3 CRUD Operations

Each resource (projects, skills, messages) follows the same controller pattern: list (fetch all, sorted newest first), get one, create (validate and save, return 201), update (`findByIdAndUpdate` with schema validation), and delete (`findByIdAndDelete`). All mutating handlers are wrapped in try/catch blocks that return structured JSON error responses.

4.4 Frontend Implementation

The `client/src/pages` directory contains one component per route. Shared presentational pieces such as the navigation bar, project cards, and skill badges live in `client/src/components`. An `AuthContext` stores the JWT after a successful admin login and attaches it to the Authorization header of every subsequent authenticated API call.

4.5 Styling

The interface is styled entirely with Tailwind CSS utility classes, keeping the design consistent across the public site and the admin panel without hand-written CSS files.

5. TESTING

5.1 Testing Strategy

Testing was carried out manually through the browser (public site and admin panel) and through Postman (direct API requests), exercising each user flow end-to-end against a running MongoDB instance, along with boundary and validation checks against the schema rules and JWT middleware.

5.2 Test Cases

ID	Test Case	Expected Output	Result
TC-01	Load home page	Hero, skills, and featured projects render correctly	Pass
TC-02	Submit valid contact message	Message saved; success toast shown	Pass
TC-03	Submit contact message with invalid email	Validation error returned; form not submitted	Pass
TC-04	Admin login with valid credentials	JWT returned; redirected to dashboard	Pass
TC-05	Admin login with invalid credentials	401 response; error shown on form	Pass
TC-06	Access protected route without token	401 Unauthorized returned	Pass
TC-07	Create project as admin	Project created; appears in list and public site	Pass
TC-08	Update existing project	Project updates; reflected on public projects page	Pass
TC-09	Delete a project	Project removed from list and database	Pass
TC-10	Create skill with colour selection	Skill saved with matching colour badge	Pass

6. RESULTS AND DISCUSSION

6.1 Output Screens

The completed application includes the following screens, to be inserted here as screenshots in the final submitted copy: Home Page (hero, skills, featured projects), Projects Page (full grid), Contact Page (form with success toast), Admin Login Page, and the Admin Dashboard with its Projects, Skills, and Messages tabs.

6.2 Discussion

The project successfully demonstrates that a decoupled React frontend and Express REST API backend can together deliver a fully functional, database-backed content management system. It reinforces a solid understanding of RESTful API design, stateless JWT authentication, and separation of concerns between the presentation layer and the data/business-logic layer. Structuring the database as three independent, non-relational collections also shows that a document-oriented approach fits naturally when the underlying data has no meaningful cross-collection relationships.

7. ADVANTAGES AND LIMITATIONS

7.1 Advantages

- Content (projects, skills) can be updated live through the admin panel without any code change or redeployment.
- Clear separation of concerns between the React frontend and Express REST API backend.
- Stateless JWT authentication avoids server-side session storage, simplifying scaling.
- Data integrity enforced at both the schema level (Mongoose validation) and the UI level.

7.2 Limitations

- Single-admin authentication only — no role-based permissions or multi-user support.
- No pagination — all projects and skills are fetched in a single request.
- Images are referenced by URL rather than uploaded and hosted directly.
- No automated test suite — testing was performed manually via the browser and Postman.

8. FUTURE ENHANCEMENTS

6. Add pagination to the projects list and messages inbox for scalability.
7. Replace URL-based images with direct image upload (e.g., via Cloudinary).
8. Add project categories and filtering/search on the public projects page.
9. Add refresh-token support to extend JWT sessions securely.
10. Add email notifications to the admin when a new contact message is submitted.

9. CONCLUSION

This project successfully delivers Portfolio, a fully functional, full-stack personal portfolio website built with React, Vite, Tailwind CSS, Express, MongoDB, Mongoose, and JWT authentication. It fulfils all specified functional requirements, including public browsing of projects and skills, contact form submission, and a secure admin panel with full CRUD management of projects, skills, and messages, while maintaining a clean separation between the client-side application and the server-side API. The resulting application is maintainable, extensible, and provides a solid foundation for the future enhancements outlined above.

10. REFERENCES

11. React Documentation — <https://react.dev/>
12. Vite Documentation — <https://vitejs.dev/>
13. Tailwind CSS Documentation — <https://tailwindcss.com/docs>
14. Node.js Documentation — <https://nodejs.org/docs>
15. Express.js Documentation — <https://expressjs.com/>
16. MongoDB Documentation — <https://www.mongodb.com/docs/>
17. Mongoose Documentation — <https://mongoosejs.com/docs/>
18. JSON Web Tokens (JWT) Introduction — <https://jwt.io/introduction>
19. MDN Web Docs — <https://developer.mozilla.org/>

11. SOURCE CODE

Deployment Links:

- Frontend: <https://bipin-pandey.vercel.app>
- Backend API: <https://bipin-portfolio-rgy9.onrender.com>
- GitHub Repository: https://github.com/tusharcodes8/bipin_portfolio.git

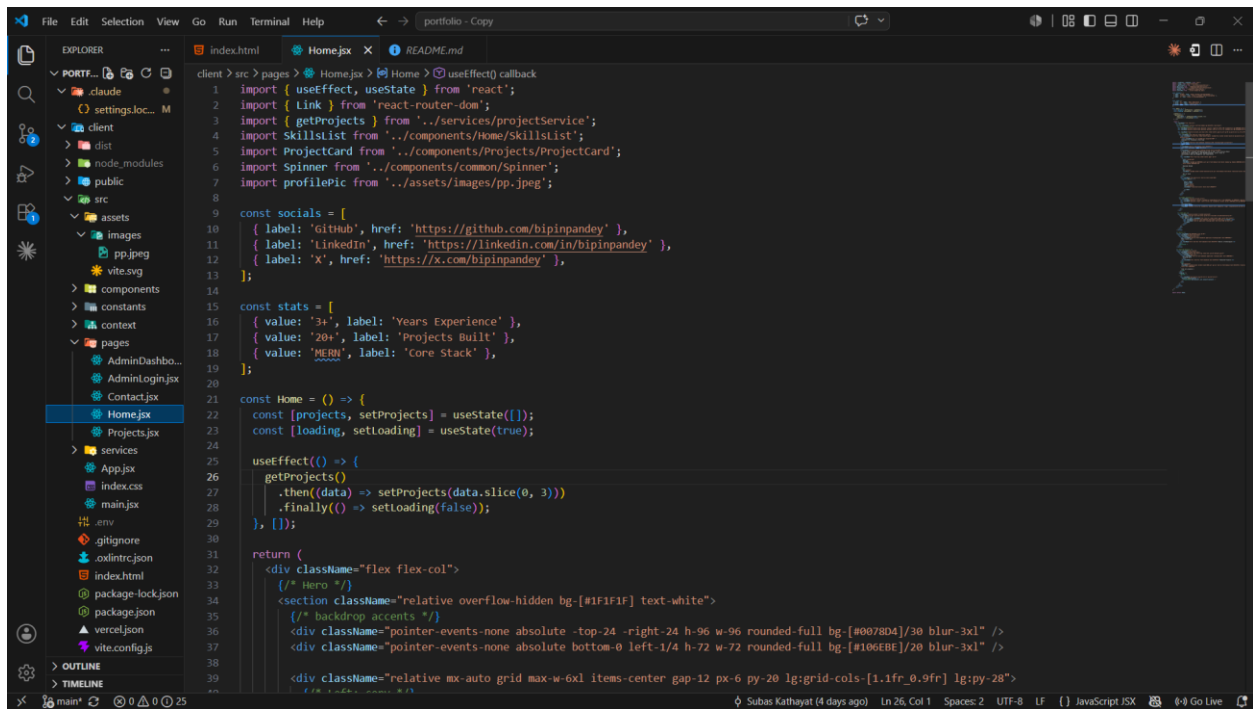
Admin Portal:

Extra links : <https://bipin-pandey.vercel.app/admin/login>

Username: admin

Password : BipinAdmin@2024

Hosting note: Vercel is used for the static/SPA frontend build, Render hosts the Node/Express backend, and MongoDB Atlas provides the managed database.



```
client > src > pages > Home.jsx > Home > useEffect() callback
1 import { useEffect, useState } from 'react';
2 import { Link } from 'react-router-dom';
3 import { getProjects } from '../services/projectService';
4 import SkillList from '../components/Home/SkillList';
5 import ProjectCard from '../components/Projects/ProjectCard';
6 import Spinner from '../components/common/Spinner';
7 import profilePic from '../assets/images/pp.jpeg';
8
9 const socials = [
10   { label: 'Github', href: 'https://github.com/bipinpandey' },
11   { label: 'LinkedIn', href: 'https://linkedin.com/in/bipinpandey' },
12   { label: 'X', href: 'https://x.com/bipinpandey' },
13 ];
14
15 const stats = [
16   { value: '3+', label: 'Years Experience' },
17   { value: '20+', label: 'Projects Built' },
18   { value: 'MERN', label: 'Core Stack' },
19 ];
20
21 const Home = () => {
22   const [projects, setProjects] = useState([]);
23   const [loading, setLoading] = useState(true);
24
25   useEffect(() => {
26     getProjects().then(data => setProjects(data.slice(0, 3)))
27       .finally(() => setLoading(false));
28   }, []);
29
30   return (
31     <div className="flex flex-col">
32       </* Hero */>
33       <section className="relative overflow-hidden bg-[#1F1F1F] text-white">
34         </* backdrop accents */>
35         <div className="pointer-events-none absolute -top-24 -right-24 h-96 w-96 rounded-full bg-[#0078D4]/30 blur-3xl" />
36         <div className="pointer-events-none absolute bottom-0 left-1/4 h-72 w-72 rounded-full bg-[#106E8E]/20 blur-3xl" />
37
38         <div className="relative mx-auto grid max-w-6xl items-center gap-12 px-6 py-20 lg:grid-cols-[1.1fr_0.9fr] lg:py-28">
```

